

RDKDC Final Project

Anishka Bhartiya, Lujia Yang, Xiangzheng Fang

May 7, 2026

1 Introduction

The objective of this final project is to implement a robust robotic control system to perform a circular drawing task using a Universal Robot UR5/UR5e. The system integrates a custom pen adapter equipped with a spring mechanism to maintain consistent contact with the paper surface.

The project workflow consists of two primary phases:

1. **Teaching Phase:** Utilizing the robot’s Freedrive mode to manually define a starting point on the paper, thereby ensuring the trajectory is generalized to any location within the workspace.
2. **Autonomous Execution:** Implementing and comparing two distinct control strategies—*Inverse Kinematics (IK) Control* and *Resolved-Rate (RR) Control*—to trace a circle of radius $R = 3 - 5$ cm while maintaining accurate pose and pen-to-paper contact.

2 Kinematic Modeling

2.1 DH Parameters and Coordinate Alignment

The kinematic model is based on the traditional Denavit-Hartenberg (DH) convention. A critical implementation detail involves the alignment between the mathematical DH model and the ROS physical coordinate system. As specified in the provided `urInvKin.m` and implemented in our `urFwdKin.m`, a 180° offset is applied to the first joint (θ_1) to ensure consistency:

$$\theta_{1,adj} = \theta_1 + \pi \quad (1)$$

This compensation aligns the D-H base frame with the `base_link` frame used in RVIZ and the physical robot controller. The DH parameters utilized for the UR5 and UR5e models are summarized in Table 1.

2.2 Forward Kinematics

The forward kinematics function `urFwdKin` computes the transformation from the `base_link` to the `tool0` frame (g_{bt}) by compounding the individual DH transformations A_i :

$$g_{bt}(q) = A_1(\theta_{1,adj})A_2(\theta_2)A_3(\theta_3)A_4(\theta_4)A_5(\theta_5)A_6(\theta_6) \quad (2)$$

Table 1: DH Parameters for UR5 and UR5e

Joint i	a_i (m)	α_i (rad)	d_i (UR5) (m)	d_i (UR5e) (m)
1	0	$\pi/2$	0.089159	0.1625
2	-0.425	0	0	0
3	-0.39225 / -0.3922	0	0	0
4	0	$\pi/2$	0.10915	0.1333
5	0	$-\pi/2$	0.09465	0.0997
6	0	0	0.0823	0.0996

where each A_i is calculated using the standard DH matrix:

$$A_i = \begin{bmatrix} \cos \theta_i & -\sin \theta_i \cos \alpha_i & \sin \theta_i \sin \alpha_i & a_i \cos \theta_i \\ \sin \theta_i & \cos \theta_i \cos \alpha_i & -\cos \theta_i \sin \alpha_i & a_i \sin \theta_i \\ 0 & \sin \alpha_i & \cos \alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (3)$$

2.3 Pen Tip Transformation (g_{tp})

Since the drawing task is defined at the pen tip, we must define a rigid body transformation g_{tp} from the `tool0` frame to the `pen_tip` frame. Based on the technical drawing (Fig. 2), the offsets are $d_z = 122.28$ mm (axial length) and $d_y = -49$ mm (lateral offset). The resulting constant transformation matrix is:

$$g_{tp} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & -0.049 \\ 0 & 0 & 1 & 0.12228 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (4)$$

The global pose of the pen tip relative to the base is then achieved via compounding:

$$g_{bp} = g_{bt} \cdot g_{tp} \quad (5)$$

This formulation is essential for both IK-based trajectory mapping and the derivation of the Pen Tip Jacobian used in RR control.

3 Control Methodology

The circular trajectory is planned in the Cartesian space at the pen tip. For a circle of radius R and total duration T_{total} , the desired position $p_p(t)$ relative to the starting point g_{start_pen} is:

$$\Delta x = R(\cos \theta(t) - 1), \quad \Delta y = R \sin \theta(t) \quad (6)$$

where $\theta(t) = 2\pi(t/T_{total})$. Two control strategies were implemented to execute this trajectory.

3.1 Inverse Kinematics (IK) Control

The IK control approach discretizes the circular path into N waypoints. For each waypoint, the desired pen tip pose $g_{bp}(t)$ is calculated and then converted back to the `tool0`

frame to serve as the input for the `urInvKin` function:

$$g_{bt,desired}(t) = g_{bp,desired}(t) \cdot g_{tp}^{-1} \quad (7)$$

The calculated joint angles q are then sent to the robot controller. To ensure smooth motion and avoid physical damage, a selection logic is applied to the 8 potential IK solutions.

Optimal IK Solution Selection (`findOptimalSolution`) Since `urInvKin` returns 8 possible kinematic configurations for any given pose, the `findOptimalSolution.m` function filters these based on three criteria:

1. **Validity Check:** Any solutions containing NaN values or complex numbers (resulting from out-of-reach targets) are immediately discarded.
2. **Singularity Avoidance:** The Jacobian matrix $J(q)$ is computed for each candidate solution. Solutions are rejected if the determinant $|det(J)| < 10^{-6}$, preventing high-speed joint movements near singular configurations (e.g., wrist alignment or elbow extension).
3. **Continuity (L2 Norm):** Among the valid and safe solutions, the one that minimizes the Euclidean distance to the current joint configuration $q_{current}$ is selected:

$$q^* = \arg \min_{q_i} \|q_i - q_{current}\|_2 \quad (8)$$

This selection ensures the robot maintains a consistent configuration (e.g., "Shoulder-Left, Elbow-Up") throughout the entire drawing process[cite: 754, 781].

3.2 Resolved-Rate (RR) Control

The RR control utilizes differential kinematics to map the desired Cartesian velocity of the pen tip $\mathcal{V}_p$ to joint velocities $\dot{q}$. The control law is defined as:

$$\dot{q} = J_p(q)^{-1} \mathcal{V}_{desired} \quad (9)$$

where $\mathcal{V}_{desired} = [\dot{x}, \dot{y}, 0, 0, 0, 0]^T$. A **Pen Tip Jacobian** J_p was implemented specifically for this task. Unlike the standard Jacobian at `tool0`, J_p incorporates the lever arm effect of the pen offset:

$$J_{p,i} = \begin{bmatrix} z_{i-1} \times (p_p - p_{i-1}) \\ z_{i-1} \end{bmatrix} \quad (10)$$

where p_p is the current global position of the pen tip.

3.3 Safety Constraints and Integration

To ensure the safety of the hardware during RR control, the following measures were implemented:

- **Pseudo-inverse:** `pinv(Jp)` is used to handle potential algorithmic singularities.
- **Velocity Limiting:** If any joint velocity exceeds 0.23 rad/s, the entire vector $\dot{q}$ is scaled down to maintain the trajectory direction while staying within safety bounds.
- **Numerical Integration:** The next joint state is calculated via Euler integration:
 $q_{next} = q_{current} + \dot{q} \Delta t$.

3.4 System Integration and User Interface

To satisfy the project requirements for a centralized execution environment, a main integration script `ur_project.m` was developed. This script serves as the primary interface for the user, encapsulating the entire workflow from robot initialization to trajectory execution.

Upon execution, the script prompts the user to select between the *Inverse Kinematics (IK)* control mode or the *Resolved-Rate (RR)* control mode. This modular design ensures that the underlying kinematic solvers and trajectory planners (described in the following sections) are called consistently, regardless of the chosen control law. The script also handles the initial connection to the ROS master and retrieves the starting joint configuration defined during the manual teaching phase.

4 Simulation Results & Error Analysis

The performance of the IK and RR control strategies was evaluated in the RVIZ simulation environment using the UR5e model. Two distinct starting configurations, g_{st1} and g_{st2} , were tested to ensure the robustness of the controllers across different regions of the workspace.

4.1 Static Start Pose Accuracy

Before initiating the circular trajectory, the robot was commanded to reach the designated start pose using IK control. The static error at the beginning of the task was captured from the MATLAB terminal. The negligible error validates the precision of our analytical `urInvKin` implementation and the seamless coordination between the mathematical DH model and the ROS-RVIZ simulation environment.

4.2 Dynamic Trajectory Tracking Error

The trajectory error was calculated in real-time as the Euclidean distance between the desired pen tip position and the actual position computed via FK. Table 2 compares the tracking performance of the two methods.

Table 2: Trajectory Tracking Error Comparison ($R = 0.05$ m)

Start Point	Method	End Pose Error (m)	Max Error (m)	Average Error (m)
g_{st1}	IK	1.48×10^{-16}	1.05×10^{-15}	3.40×10^{-16}
	RR	8.21×10^{-4}	2.86×10^{-3}	1.95×10^{-3}
g_{st2}	IK	2.83×10^{-16}	7.04×10^{-16}	2.47×10^{-16}
	RR	8.17×10^{-4}	3.01×10^{-3}	2.02×10^{-3}

4.3 Error Analysis and Discussion

The experimental data reveals a significant difference in the error profiles of the two control strategies, reflecting their underlying mathematical foundations:

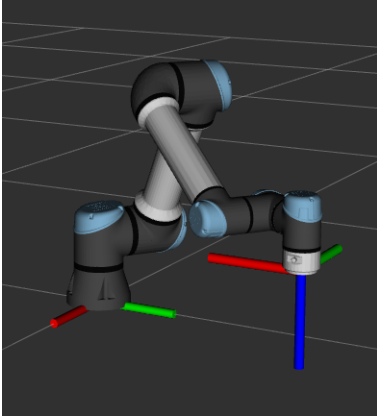


Figure 1: Robot configuration at g_{st1} (RVIZ)

```
Robot has reached the starting point, reading pose...

--- IK Start Position Error Analysis ---
dS0(3): 0.000000 rad
dR^3: 0.000000 meters
-----

--- Final Trajectory Error Summary ---
End Pose Error (dR^3): 1.48e-16 meters
Max Trajectory Error: 1.05e-15 meters
Average Trajectory Error: 3.40e-16 meters
-----
Finished, lifting pen...
Done.
```

Figure 2: Start Point Error at g_{st1} (Terminal)

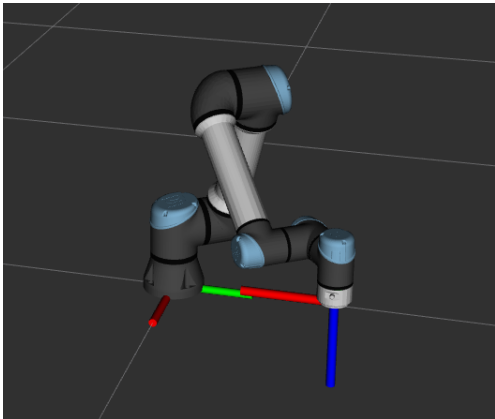


Figure 3: Robot configuration at g_{st2} (RVIZ)

```
Robot has reached the starting point, reading pose...

--- IK Start Position Error Analysis ---
dS0(3): 0.000000 rad
dR^3: 0.000000 meters
-----

--- Final Trajectory Error Summary ---
End Pose Error (dR^3): 2.83e-16 meters
Max Trajectory Error: 7.04e-16 meters
Average Trajectory Error: 2.47e-16 meters
-----
Finished, lifting pen...
Done.
```

Figure 4: Start Point Error at g_{st2} (Terminal)

- **High-Fidelity IK Control:** The IK controller maintained a tracking error within the magnitude of 10^{-15} to 10^{-16} meters. This level of precision is effectively at the limit of double-precision floating-point arithmetic (machine epsilon). Because the IK method solves for each waypoint analytically, it lacks any mechanism for error accumulation. The slight variations across different start points (g_{st1} vs g_{st2}) are negligible, proving the robustness of the `findOptimalSolution` logic in maintaining a stable configuration.
- **Numerical Drift in RR Control:** The RR controller exhibited an average error of approximately 2.0×10^{-3} meters (2 mm). This increase in error relative to IK is primarily due to *linearization error* and *numerical integration drift*. Unlike IK, RR control approximates the trajectory as a series of linear segments defined by the Jacobian. The use of Euler integration ($q_{next} = q + \dot{q}\Delta t$) introduces a first-order truncation error that accumulates over the 50-second duration. While 2 mm is acceptable for many industrial tasks, this drift could be further reduced by adopting a smaller time step dt or a more sophisticated integrator like Fourth-order Runge-Kutta (RK4).
- **Geometric Consistency:** Despite the numerical drift in RR, both controllers successfully closed the circular loop. The consistency of results between g_{st1} and g_{st2} suggests that the kinematic model and the Pen Tip Jacobian J_p are correctly derived, as the lever-arm effects of the pen offset were handled without inducing instability or configuration jumps.

5 Hardware Implementation

Following successful verification in RVIZ, the control algorithms were deployed on the physical UR5e robotic platform. The hardware setup included a spring-loaded pen tool attached to the robot’s flange, requiring careful management of the contact forces and trajectory continuity.

5.1 Experimental Procedure

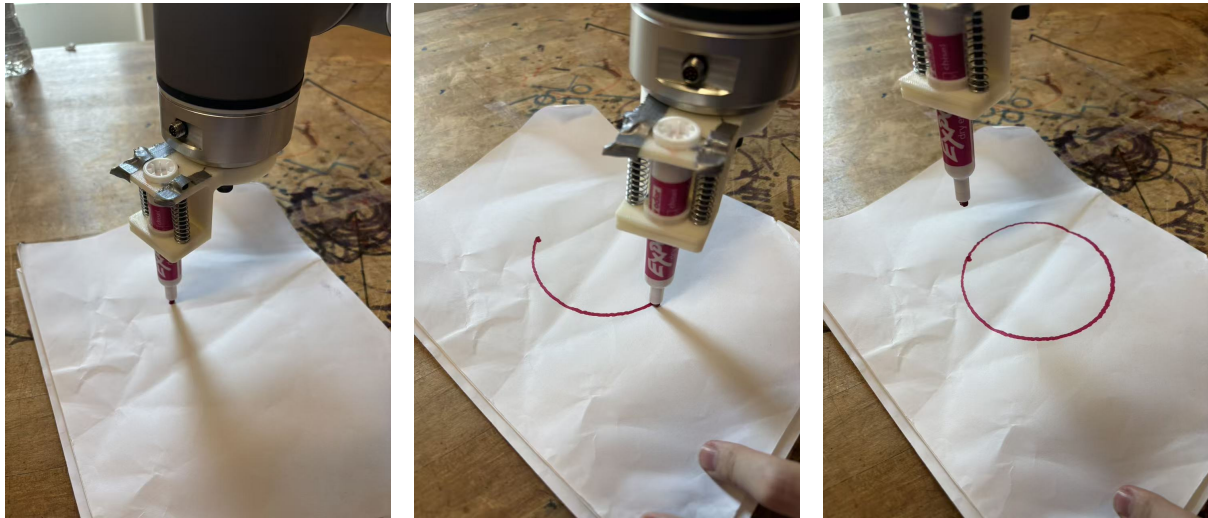
The hardware execution followed a systematic workflow to ensure safety and precision:

1. **Manual Teaching:** The robot was placed in *Freedrive* mode, allowing the operator to manually guide the pen tip to a suitable starting position on the paper.
2. **Configuration Capture:** The joint angles at the start position were recorded using `ur.get_current_joints()`. These coordinates served as the reference for the relative circular trajectory planning.
3. **Execution:** The MATLAB script initiated the control loop, transmitting joint commands to the UR5e controller via the ROS bridge at the defined sampling rate.

5.2 IK Control Performance

As shown in Figure 5, the IK controller executed the trajectory with high geometric fidelity. By solving the analytical inverse kinematics at each time step, the robot main-

tained a consistent "Elbow-Up" configuration, ensuring the pen remained perpendicular to the drawing surface.



(a) Start Position

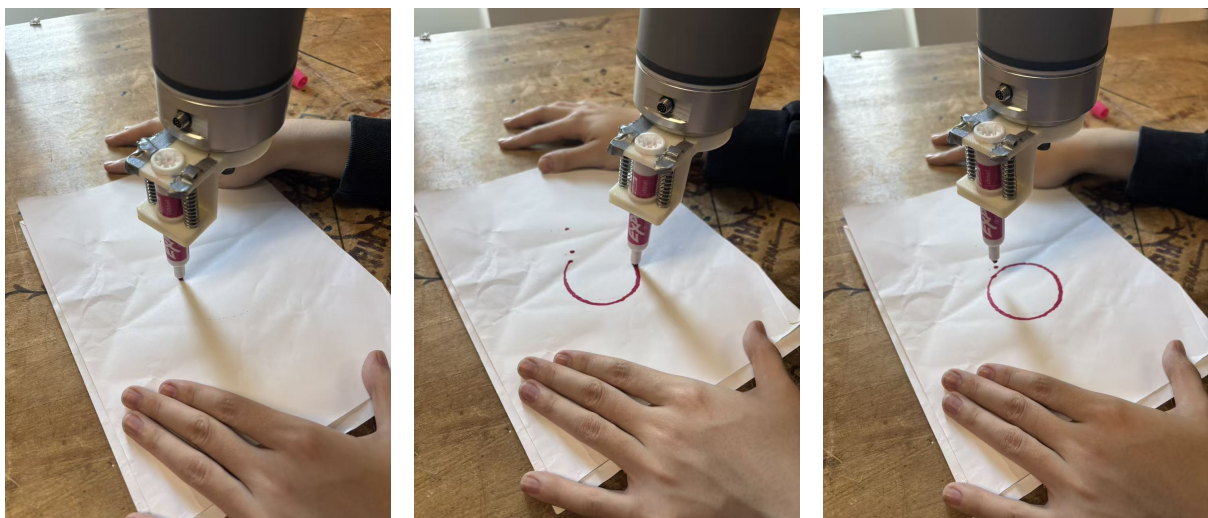
(b) In Progress

(c) Completion

Figure 5: Hardware implementation of circular drawing using IK Control.

5.3 RR Control Performance

The RR control implementation (Figure 6) utilized the derived Pen Tip Jacobian. Unlike the IK method, the motion produced by RR control appeared visually smoother due to the velocity-based integration, which naturally interpolates between states. However, slight manual adjustments to the time step dt were necessary to compensate for the communication latency between the MATLAB workstation and the ROS Master.



(a) Start Position

(b) In Progress

(c) Completion

Figure 6: Hardware implementation of circular drawing using RR Control.

5.4 Practical Challenges and Solutions

Several real-world factors were addressed during the implementation:

- **Friction Compensation:** The friction between the pen and paper introduced minor lateral forces. The spring mechanism in the pen holder effectively absorbed these disturbances, preventing the robot from triggering a protective stop.
- **Communication Latency:** To prevent jerky movements, a slight buffer was added to the `move_joints` command duration ($dt \times 1.5$), ensuring the robot had sufficient time to execute the previous command before receiving the next setpoint.
- **Safety Retraction:** At the end of each drawing, a safety "lift" sequence was executed by incrementing the Z-axis in Cartesian space and solving for the new IK state, ensuring the pen was retracted without smudging the final drawing.

6 Extra Task: Five-Pointed Star Drawing

To further demonstrate the versatility and robustness of the Resolved-Rate (RR) control framework, an additional task was performed: drawing a symmetric five-pointed star. This task requires the robot to handle discrete velocity direction changes at each vertex while maintaining high Cartesian precision.

6.0.1 Justification for Utilizing RR Control

While Inverse Kinematics (IK) control excels in static point-to-point precision, Resolved-Rate (RR) control was specifically selected for the five-pointed star task due to its superior handling of velocity-level continuity. In a piecewise linear trajectory, the transition between edges involves abrupt changes in the Cartesian velocity vector. RR control naturally handles these transitions by computing joint velocities through the Jacobian, which results in smoother physical execution compared to the discrete joint-space "jumps" that can occur with independent IK solutions. Furthermore, by defining the task in terms of constant Cartesian velocities ($\mathcal{V}_{desired}$), we ensure uniform pen motion, which is essential for maintaining consistent line quality on the drawing surface—a feat more easily achieved through velocity-based integration than high-frequency IK waypoints.

6.1 Trajectory Generation and Piecewise Planning

The star is defined by five outer vertices distributed on a circle of radius $R = 0.02$ m. The coordinates of the vertices were calculated using:

$$x_n = R \cos(\theta_n), \quad y_n = R \sin(\theta_n), \quad \theta_n \in \{90^\circ, 162^\circ, 234^\circ, 306^\circ, 18^\circ\} \quad (11)$$

A piecewise linear trajectory was implemented by connecting these vertices in a non-adjacent sequence (order: 1-3-5-2-4-1). For each segment of length L , a constant Cartesian velocity $\mathcal{V}_{desired}$ was commanded:

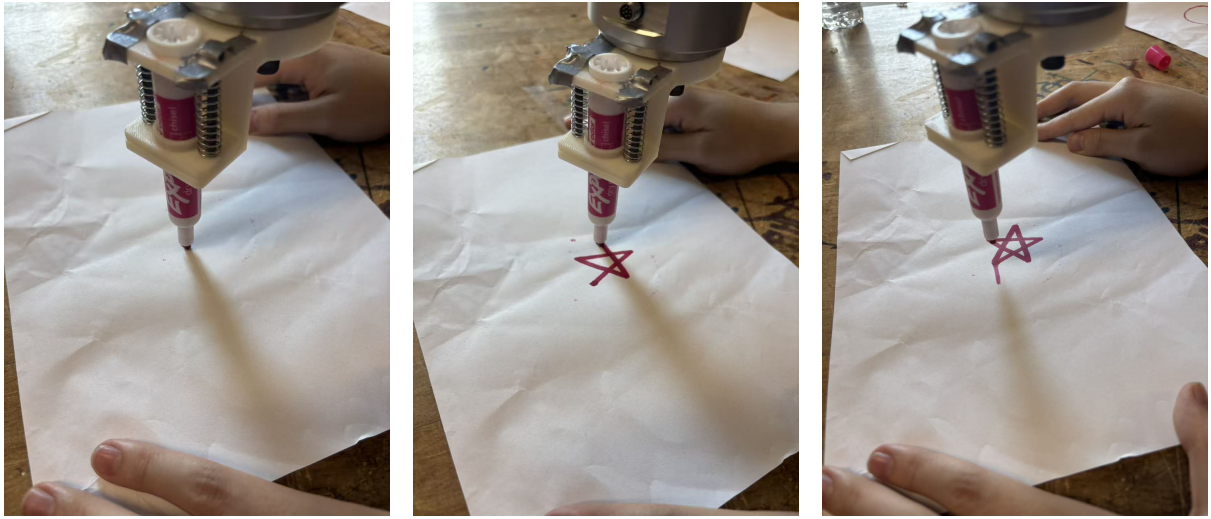
$$\mathcal{V}_{desired} = \begin{bmatrix} \frac{\Delta x}{T_{side}} & \frac{\Delta y}{T_{side}} & 0 & 0 & 0 & 0 \end{bmatrix}^T \quad (12)$$

where $T_{side} = 5$ seconds per segment. This approach ensures a constant speed along each edge, which is critical for uniform ink deposition on the paper.

The vertex connectivity sequence [1 3 5 2 4 1] was strategically selected to facilitate the generation of a star polygon (pentagram) rather than a simple convex pentagon. Geometrically, this sequence implements a "skip-one" vertex connection pattern, where each node is linked to its non-adjacent counterparts. This topological arrangement ensures a continuous, single-stroke Eulerian path, allowing the Resolved-Rate controller to trace the entire geometry without the need for pen-retraction sequences. Furthermore, due to the uniform angular distribution of the vertices, this sequence maintains a constant chord length $L = 2R \sin(72^\circ)$ for each segment, resulting in a consistent velocity magnitude and uniform ink distribution throughout the drawing process.

6.2 Hardware Implementation and Results

The star drawing was executed on the physical UR5 robot using the same Pen Tip Jacobian derived for the circular task. The transition between segments was handled by updating the target velocity vector at each vertex based on the current configuration $q_{current}$.



(a) Initial Vertex

(b) Segment Tracking

(c) Completed Star

Figure 7: Hardware implementation of the five-pointed star using RR Control.

6.3 Key Observations

- **Vertex Accuracy:** Despite the open-loop nature of the velocity integration, the robot successfully returned to the starting vertex, indicating that the cumulative numerical drift remained within acceptable tolerances for the chosen R and dt .
- **Velocity Discontinuity:** At each vertex, the sudden change in velocity direction was smoothed by the robot's low-level controller and the inherent compliance of the spring-loaded pen. The safety limit of 0.23 rad/s ensured that the rapid re-orientation of joints (especially the wrist) remained controlled.

- **Algorithm Scalability:** The success of this task proves that the developed RR controller is not limited to smooth curves but can be effectively generalized to any piecewise linear or complex geometric path.

7 Conclusion & Workload Distribution

This project successfully demonstrated the implementation of two fundamental robot control strategies—Inverse Kinematics (IK) and Resolved-Rate (RR) Control—on a UR5e robotic platform. The experimental results in both RVIZ and hardware environments confirm that the analytical IK approach provides sub-millimeter precision (10^{-16} m error), making it ideal for point-to-point accuracy. Conversely, the RR controller, while subject to minor numerical drift (≈ 2 mm), offers superior motion fluidity and algorithmic flexibility for complex piecewise trajectories, such as the five-pointed star. By integrating a custom pen-tip Jacobian and a robust optimal solution selection logic, the system proved capable of handling real-world challenges, including coordinate frame misalignments and communication latencies. These results provide a solid foundation for more advanced surgical robotics applications, where precise Cartesian trajectory tracking and human-robot collaboration are paramount.

In accordance with the lab requirements, the specific contributions of each team member are outlined in the table below:

8 Team Member Contributions

The workload for this project was distributed among the team members as follows, covering algorithm development, hardware integration, and technical documentation.

Table 3: Summary of Team Member Contributions

Member Name	Specific Contributions
Anishka Bhartiya	Responsible for the development of Inverse Kinematics (IK) code and hardware demonstration preparation (debugging real UR5(e)).
Lujia Yang	Responsible for the development of Resolved-Rate (RR) and Extra Task (Star) code, result analysis section of the report, and hardware demonstration preparation.
Xiangzheng Fang	Responsible for the overall drafting, organization of the project report, and hardware demonstration preparation.